LibraOS: An Interactive Operating-System Concepts Simulator

1st Prerana Ghagare
Department of Computer Science and Engineering
St. Vincent Pallotti College of Engineering and Technology
Nagpur, India
preranaghagare.25d@stvincentngp.edu.in

2nd Gayatri Bute
Department of Computer Science and Engineering
St. Vincent Pallotti College of Engineering and Technology
Nagpur, India
gayatribute.25d@stvincentngp.edu.in

[Submission Date]

Abstract— Operating-systems concepts are often introduced through algorithms and data structures that are difficult to observe as a connected system. LibraOS is a browser-based teaching simulator that brings these ideas together in a library-management setting. A JavaScript desktop communicates over WebSocket with a Python FastAPI service, which coordinates a simulated kernel, inode-and-block file system, process manager, and library application. Interactive modules demonstrate CPU scheduling, virtual-memory page replacement, disk-head scheduling, synchronization, deadlock handling, access control, system calls, and journal recovery. A persistent JSON disk image, deterministic workloads, bounded request traces, and replayable event archives support repeatable demonstrations. Evaluation consists of the repository's automated functional tests: all 103 Python unit tests and all 4 Node.js session-archive tests passed in the reported local run. These results establish tested behavior, not performance or equivalence to a production operating system; CPU, memory, identity, and concurrency are modeled for education. LibraOS therefore provides a coherent, inspectable environment for exploring operating-system mechanisms while making the boundary between simulation and host behavior explicit.

Index Terms— operating systems education, simulation, CPU scheduling, file systems, virtual memory, synchronization, observability.

I. INTRODUCTION

An operating system coordinates processes, memory, persistent storage, devices, and access to shared resources. These responsibilities are naturally separated into topics when taught, but students benefit from seeing how the topics interact in a running system [1]–[3]. LibraOS addresses this instructional challenge with an interactive desktop whose applications expose a simulated kernel through visual traces, workload editors, and a library-management workflow.

The project is not a bootable operating system and does not take control of host hardware. Instead, it implements selected operating-system abstractions as deterministic software models. Users can inspect simulated file-system blocks, compare scheduler outputs, follow page faults, observe resource wait queues, and perform circulation actions against records stored on the virtual disk. This shared scenario gives abstract mechanisms a familiar context: searches, member lookups, imports, checkouts, and reports act as workloads for system components.

This paper describes LibraOS's architecture, implemented models, persistence and observability design, and functional evaluation. It also discusses the boundaries of the simulator so that its demonstrations are not mistaken for measurements of, or protections provided by, a real operating system.

II. BACKGROUND AND DESIGN OBJECTIVES

Operating-systems texts describe abstractions such as processes, scheduling policies, address spaces, synchronization primitives, and file allocation as mechanisms for managing finite resources [1]–[4]. LibraOS translates a selection of those mechanisms into discrete simulations with visible state and explicit outputs. Scheduling policies produce per-core timelines and process metrics; replacement policies expose hit, fault, and eviction traces; disk policies show request order and head movement.

The design follows four objectives. First, keep the user interface separate from kernel logic: browser requests carry structured data, and the server translates them into operations on simulated components. Second, favor inspectable and repeatable demonstrations over hidden state: workloads and scenarios are deterministic, and results include timelines, counters, or histories. Third, keep library records within the simulated file system instead of using host files as application records. Fourth, state limitations clearly: demo accounts are not authentication, and resource indicators are not measurements of the host.

III. SYSTEM ARCHITECTURE

The browser presents a desktop shell with a terminal, File Explorer, Task Manager, Sync Monitor, Resource Monitor, Security, Library, Trace & Journal, and Scenario Lab. The front end is built with HTML, CSS, and JavaScript modules. It sends JSON requests and receives JSON results and kernel events through a FastAPI WebSocket endpoint. The server hosts the static interface, a health endpoint, the WebSocket adapter, and the kernel lifecycle.

Within the kernel, specialized managers implement the modeled responsibilities. The scheduler operates on process workloads; the memory manager maps simulated pages to a shared set of frames; the disk scheduler orders track requests; and the synchronization manager tracks resource ownership, blocked processes, and deadlocks. The security and syscall modules check demo-role permissions and record modeled system calls. The library manager reads and updates book, member, issue, and fine records through the simulated file system. A bounded event hub distributes status events to connected clients.

The virtual disk is a JSON image containing a superblock, inode table, free-block bitmap, and Base64-encoded block contents. Its default configuration has 512 blocks of 256 bytes. Files use contiguous, linked, or indexed allocation. Disk-image saves use atomic replacement; mutating transactions write a redo intent and then persist the new image before recording a commit marker. On boot, an intent without a commit marker can be replayed. Request traces correlate operation identifiers, outcomes, and durations, while journal records describe persistence and recovery.

[RESERVED SPACE FOR IMAGE]

Insert a LibraOS screenshot, system architecture diagram, or other project image here.

Fig. 1. Project image placeholder. Replace this box with a selected screenshot or diagram.

IV. IMPLEMENTED SIMULATION MODULES

A. Processes and CPU scheduling

The process manager maintains process-control-block state and supports lifecycle operations such as fork, exec, wait, kill, and priority changes. The scheduler validates each workload before simulation and supports First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Round Robin, preemptive and non-preemptive priority, and Multilevel Feedback Queue (MLFQ) [3], [4]. Workloads can run across one to four simulated cores. Results include per-core Gantt intervals, ready-queue snapshots, waiting, turnaround, and response times, context switches, utilization, and throughput. These are outputs of the modeled workload, not operating-system measurements.

B. File systems, memory, and disk scheduling

The inode-based file system separates file metadata from data blocks and exposes allocation maps and permissions in File Explorer, illustrating classic file-system abstractions [1], [3], [4]. Seed data includes 60 book records, 25 members, 20 issue records, and 8 overdue-fine records. The library application uses these JSON files for catalog and circulation operations. Atomic disk-image saves and redo recovery make persistence behavior inspectable, while the journal enables demonstrations of transaction recovery.

The memory simulator creates per-process virtual address spaces and uses eight shared physical frames by default. FIFO, least-recently-used (LRU), and Optimal replacement models report page references, faults, hits, and evictions [3], [6]. The disk scheduler compares FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK policies using service order and total head movement [3], [4]. Its track movement is simulated and must not be interpreted as physical storage-device latency.

C. Synchronization, security, and observability

Mutexes and counting semaphores track allocations, requests, and FIFO wait queues; an unsatisfied request blocks its simulated process [2], [3], [5]. Deadlock detection examines the modeled resource-allocation state, and recovery releases resources held by a selected process. A fixed interleaving illustrates a lost update and compares it with mutex-serialized updates. This is a deterministic concurrency demonstration rather than a race between host threads.

Security exercises role-based checks for guest, librarian, and administrator demo identities, and the syscall tracer records modeled calls. The accounts have no passwords and provide no real authentication. The Session Studio exports a bounded event archive containing allowlisted metadata; imported archives are replayed visually and do not execute actions. Scenario Lab runs repeatable comparisons without changing the live process table.

Component	Concept made visible	Representative output
Task Manager	CPU scheduling and process lifecycle	Gantt lanes, queues, process metrics
Resource Monitor	Page replacement and disk scheduling	Frame state, faults, request path, movement
Sync Monitor	Mutual exclusion and deadlock	Ownership, wait queues, recovery, race trace
File Explorer / Library	Inodes, blocks, persistence, circulation	Allocation maps, records, journal history
Security / Trace & Journal	Authorization and observability	Access decisions, syscall and request events

V. EVALUATION AND RESULTS

A. Method

Evaluation used the repository's automated functional checks, rather than a hardware benchmark. The Python suite was run with python -m unittest discover -s tests -v, covering file-system operations, scheduling, synchronization, resources, security, the library, observability, and server behavior. The JavaScript archive checks were run with node --test tests/test_stage9_session.mjs. Temporary disk images are used by tests that exercise persistence and journal recovery.

B. Results

In the reported local run, the Python suite completed with 103 tests passed and no failures. The Node.js session-archive suite completed with 4 tests passed and no failures. Collectively, these checks exercise the correctness of selected state transitions, validation paths, API behavior, persistence/recovery cases, and archive handling under test inputs. The test totals are functional evidence only: they do not establish benchmark performance, usability outcomes, production readiness, or complete coverage of every interaction.

VI. DISCUSSION AND LIMITATIONS

LibraOS's main contribution is the connection among interactive visualizations, kernel-like state, and a single application domain. A learner can move from a circulation workload to a scheduler comparison, inspect persistent files, and then examine related request and journal events. Deterministic scenarios make comparisons reproducible, while trace output makes internal state changes available for discussion. Separating browser, server, and kernel modules also gives students clear boundaries for extending or testing the project.

Interpretation requires care. All processes, cores, frames, locks, disk tracks, and scheduling times belong to a software model. Top-bar CPU and memory figures are presentation placeholders. The security module demonstrates authorization logic but does not authenticate users; accounts do not use passwords. The application is intended for local teaching and demonstration, not deployment as a secure multi-user service on an untrusted network. Browser automation and testing on every supported environment are additional validation activities, not represented by the local functional-test totals reported above.

VII. CONCLUSION

LibraOS implements an interactive, library-themed environment for demonstrating operating-system abstractions. Its browser desktop, FastAPI interface, simulated kernel, virtual disk, and visual modules connect scheduling, file systems, memory, synchronization, security, and observability in one project. Automated tests provide evidence for selected functional behavior, while explicit disclosures prevent simulated resources and demo access checks from being mistaken for host behavior or production security. Future instructional evaluation could measure how learners use the visualizations and compare comprehension before and after guided exercises; such learning outcomes are not established by the current software tests.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA, USA: Pearson, 2015.

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[4] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Pearson, 2018.

[5] E. W. Dijkstra, “Cooperating sequential processes,” in Programming Languages, F. Genuys, Ed. London, U.K.: Academic Press, 1968, pp. 43–112.

[6] L. A. Belady, R. A. Nelson, and G. S. Shedler, “An anomaly in space-time characteristics of certain programs running in a paging machine,” Communications of the ACM, vol. 12, no. 6, pp. 349–353, 1969.

LibraOS is an educational simulator; modeled resource values are not measurements of the host computer.